

ПРАКТИЧЕСКОЕ ЗАДАНИЕ №4

Классические методы поиска линий

1. ЦЕЛЬ РАБОТЫ

Целью практического задания является освоение и анализ алгоритмов детектирования линий в изображениях, применяемых для решения задач компьютерного зрения, калибровки камер и анализа структуры сцены. В работе изучаются последовательные этапы обработки: выделение границ объектов методом Кэнни, извлечение прямолинейных структур с помощью преобразования Хафа и устойчивое определение точки схода с помощью метода RANSAC. Практическая часть направлена на развитие навыков применения данных методов в реальных сценариях.

2. ОСНОВНЫЕ ТЕОРЕТИЧЕСКИЕ ПОЛОЖЕНИЯ

Рассмотрим методы, обеспечивающие надёжную обработку и анализ структуры изображения, позволяющие реализовать точную калибровку камеры и построение пространственной модели сцены для сложных задач компьютерного зрения.

2.1. Метод детектирования границ Кэнни

Детектор Кэнни — классический алгоритм выделения границ, используемый для поиска резких перепадов яркости, соответствующих границам объектов на изображении.

Основные этапы детектора Кэнни:

1. Сглаживание изображения фильтром Гаусса (для подавления шума).
2. Вычисление градиента интенсивности по осям x и y .

Для каждого пикселя (i, j) вычисляют производные:

$$G_x(i, j) = \frac{I(i+1, j) - I(i, j) + I(i+1, j+1) - I(i, j+1)}{2}$$
$$G_y(i, j) = \frac{I(i, j+1) - I(i, j) + I(i+1, j+1) - I(i+1, j)}{2}$$

Итоговый градиент и угол направления вычисляется по формуле:

$$G(i, j) = \sqrt{G_x^2 + G_y^2}$$
$$\theta(i, j) = \arccat\left(\frac{G_y}{G_x}\right)$$

3. Подавление немаксимумов (non-maximum suppression) — оставляют только

локальные максимумы градиента.

4. Двойная пороговая фильтрация и трассировка границ (hysteresis):
используются пороги T_LOW , T_HIGH ; отсеиваются шумовые и слабые границы

2.2. Преобразование Хафа

Преобразование Хафа — алгоритм поиска геометрических примитивов (линий) в бинарном изображении, устойчив к шуму и разрывам.

При прямых полярных координат обычно уравнение линии ($y = mx + b$) неудобно для вертикальных направлений, поэтому используется нормальная форма уравнения:

$$p = x \cos \theta + y \sin \theta ,$$

где p — расстояние от центра, θ — угол наклона нормали к оси x .

Алгоритм преобразования Хафа включает в себя следующие этапы:

- Создание двумерного массива $A(p, \theta)$, соответствующий комбинациям значений p, θ ;
- Для каждой точки граничного изображения вычисляют p при различных θ , инкрементируют соответствующую ячейку массива;
- Поиск максимумов в массиве с наибольшим количеством голосов;
- Восстановление линий — по найденным параметрам (p, θ) извлекаются линии на оригинальном изображении.

Алгоритм Хаффа ритуализация в OpenCV следящими образом:

`cv.HoughLines(image, rho, theta, threshold)` – стандартное преобразование Хафа
`cv.HoughLinesP(image, rho, theta, threshold)` – вероятностное преобразование Хафа

Рисунок 1 – Реализация преобразования Хафа

2.3. Метод RANSAC (Random Sample Consensus)

RANSAC — итерационный алгоритм поиска параметров модели в данных с выбросами. Позволяет устойчиво находить линии или иную структуру даже при большом количестве аномальных точек.

Минимальное число итераций для гарантии нахождения хорошей модели определяется следящим уравнением:

$$N = \frac{\log(1 - p)}{\log(1 - (1 - \epsilon)^s)}$$

Где:

- p - желаемая вероятность успеха;

- ε - предполагаемая доля выбросов;
- s - минимальное число точек для построения модели.

Алгоритм преобразования RANSAC включает в себя следующие этапы:

1. Случайным образом из данных выбирается несколько «примеров» для простейшего решения поставленной задачи;
2. Формируется решение (строится модель) по выбранным примерам;
3. Рассчитывается невязка/или число inliers исходя из заданного допуска;
4. Повторяется процедура 1-3;
5. Выбирается лучшее решение исходя из минимума невязки или максимума числа inliers.

Метод RANSAC часто применяется для поиска пересечения случайно выбранных пар линий и оценки минимальной ошибки для точки, наиболее согласованной с остальными линиями. В результате работа метода получается устойчиво определённая точки схода даже при зашумлённых данных.

3. ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ И ПРАКТИЧЕСКАЯ ЧАСТЬ

3.1. Подготовка данных

- Запишите короткое видео (6-15 секунд) с однозначно идентифицируемые параллельными линиями (тротуар с бордюром, пути и тд). Постарайтесь минимизировать тряску камеры и движений камеры в сторону.
- Убедитесь, что у вас есть:
 1. Репозиторий [cv book](#);
 2. Файл Module_5/vanishing_point.py;
 3. Ваше видео преобразовано в формат avi.

Алгоритм выполнения работы:

1. Инициализация библиотек

Перед началом работы убедитесь, что в виртуальном окружении установлены srcsam version 1.2.1, python 3.9, opencv 3.4.9. Установить данные библиотеки можно командой (рисунок 2).

```
pip install .
```

Рисунок 2 – Пример кода

2. Описание программного модуля

В программном модуле есть глобальная переменная, отвечающая за порог фильтрации для горизонтальных и вертикальных линий и подставлена на рисунке 3.

```
REJECT_DEGREE_TH = 4.0  
# Порог отклонения по углу
```

Рисунок 3 – Пример кода

В точке входа или блоке главного исполнения main происходит захвата и обработки видеок кадров. На первом этапе (рисунок 4) происходит реализация ключевых объектов: открытие видеопотока, загрузка параметров режима камеры, расчет необходимых преобразований для работы с трехмерными точками в пространстве.

Данная часть отвечает за подготовку всех технических ресурсов и параметров, необходимых для дальнейших обработок, включая правильное описание параметров камеры и сохранения их для последующих вычислений.

```
# Открываем видеопоток из файла  
cap = cv2.VideoCapture("./data/city/trm.169.007.avi")  
  
# Указываем параметры калибровки камеры  
par = ['K', 'D', 'r', 't']  
# Чтения калибровочных параметров  
calib_reader = CalibReader(file_name=os.path.abspath('./data/city/leftImage.yml'), param=par)  
calib_dict = calib_reader.read()  
# Создаем объект калибровки камеры  
calib = Calib(calib_dict)  
camera = Camera(calib)  
# Проецируем 3D-точку в 2D-изображение  
imPoint = camera.project_point_3d_to_2d(Point3d((0, 100, 0)))
```

Рисунок 4 – Пример кода

На втором этапе (рисунок 5) кода реализуется основной цикл обработки каждого видеок кадра, поступающего из видеопотока. Цикл начинается с попытки определить очередной кадр с помощью cap.read(). Если кадры закончились (f is False), цикл завершает свою работу.

Для каждого точного кадра выполняются следующие шаги:

- Детектируются основные линии изображения с помощью функции GetLines(frame), которая позволяет выделять структурные элементы сцены;

- На основании найденных линий вычисляется точка пересечений исходных линий (точка схода) с помощью `GetVanishingPoint(Lines)`. Это ключевой этап для геометрического анализа изображения;
- Если точка схода не найдена, цикл переходит к следующему кадру;
- При наличии точек пересечения происходит визуализация результатов: найденные линии рисуются зеленым, точка схода – красным кругом, а заранее сгенерированная проекция 3D-точки по полученным параметрам – синим кругом.

```
# Иницилируем переменную для цикла
f = True

# Основной цикл обработки видеопотока (работает, пока удаётся читать кадры)
while f:
    # Пытаемся считать очередной кадр из видеопотока:
    f, frame = cap.read()

    # Если видео закончилось (кадр не считан), завершить цикл
    if f is False:
        break

    # Получаем линии на текущем кадре
    Lines = GetLines(frame)

    # Находим vanishing point по найденным линиям
    VanishingPoint = GetVanishingPoint(Lines)

    # Если vanishing point не найдена, переходим к следующему кадру
    if VanishingPoint is None:
        continue

    # Отображаем найденные линии на исходном кадре (зелёным цветом)
    for Line in Lines:
        cv2.line(frame, (Line[0], Line[1]), (Line[2], Line[3]), (0, 255, 0), 2)

    # Рисуем vanishing point (красный круг)
    cv2.circle(frame, (int(VanishingPoint[0]), int(VanishingPoint[1])), 5, (0, 0, 255), -1)

    # Рисуем проекцию заранее заданной 3D-точки (синий круг)
    cv2.circle(frame, imPoint, 5, (255, 0, 0), -1)

    # Показываем финальный результат обработки для текущего кадра
    cv2.imshow("OutputImage", frame)

    # Краткая задержка для обновления окна
    cv2.waitKey(3)
```

Рисунок 5 – Пример кода

В функции `GetLines` (рисунок 6) первоначально исходное изображение преобразуется в преобразование серого для уменьшения избыточной цветной информации, затем применяется размытие по Гауссу для подавления шумов и повышения устойчивости обработки. После этого осуществляется детектирование границ методом Кэнни, что позволяет исключить резкие переходы длинных — потенциальные границы объектов.

На конечном этапе выделенные на градиентном изображении линии извлекаются с помощью преобразования Хафа, а затем производят дополнительную фильтрацию по градиентному наклону и другим характеристикам для выделения нерелевантных результатов. Итогом работы становится набор отфильтрованных прямых, наиболее информативно описывающих сцены для дальнейшего анализа задачи.

```
# Переводим изображение в оттенки серого
GrayImage = cv2.cvtColor(Image, cv2.COLOR_BGR2GRAY)

# Размываем изображение для уменьшения шумов
BlurGrayImage = cv2.GaussianBlur(GrayImage, (5, 5), 1)

# Выделяем границы на изображении (алгоритм Кэнни)
EdgeImage = cv2.Canny(BlurGrayImage, 40, 255)

# Ищем прямые линии с помощью преобразования Хафа
Lines = cv2.HoughLinesP(EdgeImage, 1, np.pi / 180, 50, 10, 15)

# Проверяем, найдены ли линии
if Lines is None:
    print("Не найдено достаточно линий для поиска точки схода.")
    exit(0)

# Фильтруем линии по углу наклона и другим параметрам
FilteredLines = FilterLines(Lines)
```

Рисунок 6 – Пример кода

В функции фильтра (рисунок 7) осуществляется отбор наиболее информативных направлений с помощью анализа их угла наклона и длины. Для каждой найденной линии вычисляются параметры уравнений прямой и угла относительно горизонта, после чего по заданному порогу отбрасываются линии, близкие к горизонтали или вертикали.

Кроме того, чтобы уменьшить повторяемость и повысить эффективность данного алгоритма, среди оставшихся отбираются только самые длинные линии, представляющие собой наиболее значимые элементы структуры изображения.

```

# Создаём список для отфильтрованных линий
FinalLines = []

# Перебираем все найденные линии
for Line in Lines:
    [[x1, y1, x2, y2]] = Line # Извлекаем координаты концов линии

    # Считаем коэффициенты прямой (y = mx + c)
    if x1 != x2:
        m = (y2 - y1) / (x2 - x1)
    else:
        m = 100000000 # Для вертикальных линий большое значение
        c = y2 - m*x2

    # Вычисляем угол наклона линии
    theta = math.degrees(math.atan(m))

    # Пропускаем горизонтальные и вертикальные линии, оставляем остальные
    if REJECT_DEGREE_TH <= abs(theta) <= (90 - REJECT_DEGREE_TH):
        l = math.sqrt((y2 - y1)**2 + (x2 - x1)**2) # Длина линии
        FinalLines.append([x1, y1, x2, y2, m, c, l])

# Если найдено слишком много линий, сохраняем только самые длинные (не более 15)
if len(FinalLines) > 15:
    FinalLines = sorted(FinalLines, key=lambda x: x[-1], reverse=True)
    FinalLines = FinalLines[:15]

# Возвращаем итоговый список линий
return FinalLines

```

Рисунок 7 – Пример кода

Функция `GetVanishingPoint` (рисунок 8) реализует вычисление точек пересечения перспективных линий (точек схода) по множеству найденных линий, с использованием градиента, аналогичного алгоритма RANSAC. Для каждой пары определяются точки их пересечения, а затем вычисляется следующая ошибка, как корень количества квадратов расстояний от этих точек до всех остальных линий.

В процессе перебора вариант с минимальной ошибкой преобразуется в качестве итоговой точки схода, как наиболее согласованной с геометрией изображения. Такой подход обеспечивает устойчивое определение точек схода даже при наличии лишних и зашумленных проводов, и часто применяется для дальнейшего отображения камеры и анализа сцены.

```

# Инициализируем переменные для точки
VanishingPoint = None
MinError = 100000000000

# Перебираем все возможные пары линий
for i in range(len(Lines)):
    for j in range(i+1, len(Lines)):
        m1, c1 = Lines[i][4], Lines[i][5] # Коэффициенты первой линии
        m2, c2 = Lines[j][4], Lines[j][5] # Коэффициенты второй линии

        # Если линии не параллельны, находим их пересечение
        if m1 != m2:
            x0 = (c1 - c2) / (m2 - m1) # Координата x точки пересечения
            y0 = m1 * x0 + c1           # Координата y точки пересечения
            err = 0

            # Считаем ошибку: суммируем квадраты расстояний от точки до каждой линии
            for k in range(len(Lines)):
                m, c = Lines[k][4], Lines[k][5]
                m_ = (-1 / m)
                c_ = y0 - m_ * x0
                x_ = (c - c_) / (m_ - m)
                y_ = m_ * x_ + c_
                l = math.sqrt((y_ - y0)**2 + (x_ - x0)**2)
                err += l**2
            err = math.sqrt(err)

            # Обновляем точку схода, если ошибка минимальна
            if MinError > err:
                MinError = err
                VanishingPoint = [x0, y0]

# Возвращаем координаты найденной точки схода
return VanishingPoint

```

Рисунок 8 – Пример кода

3. Загрузите видео и параметры для их обработки

Предварительно подготовлены исходных данных и параметров для анализа потребуется загрузить в исполняемый модуль. После исполнения модуля видео открывается с помощью программного интерфейса и становится доступным для посткадровой обработки. Пример выполнения этапа представлен на рисунке 9.



Рисунок 9 – Пример итогового видео

4. Сохранения итогового видео

Напишите код, который сохраняет результат видео в файл с помощью OpenCV, а затем предоставьте в отчете и отправьте видео преподавателю на проверку. Пример команды представлен на рисунке 10.

```
out = cv2.VideoWriter("output_video.avi", fourcc, fps, (width, height))
```

Рисунок 10 – Пример кода

5. Ответе на вопросы и предоставите в отчете:

1. Почему необходимо применять гауссово размытие перед вычислением градиента в методе Кэнни, и какую роль играет подавление немаксимумов (non-maximum suppression)?

2. Почему при использовании преобразования Хафа применяется нормальная форма уравнения линии вместо классического уравнения? Какие преимущества это дает при обработке вертикальных линий?

3. Что такое метод RANSAC и в чем состоит его главное преимущество при определении точки схода?

Литература:

- Документация OpenCV – <https://opencv.org/>
- Учебное пособие по калибровке камеры в OpenCV - <https://waksoft.susu.ru/2020/02/29/kalibrovka-kamery-s-ispolzovaniem-s-opencv/>
- Ресурсы по работе с кватернионами – <https://quaternions.online/>
- Дополнительная шпаргалка по компьютерному зрению – <https://tproger.ru/translations/opencv-python-guide>